SEES Reference and Known-Truth Validation Tutorial

Sieve Estimation of Economic Structural Models

econirl package documentation

May 2026

RTD page: https://econirl.readthedocs.io/en/latest/estimators/sees.html **Source:** https://github.com/rawatpranjal/EconIRL/tree/main

1 Purpose

SEES estimates structural dynamic discrete choice models by approximating the integrated value function with deterministic sieve basis functions. NFXP solves the Bellman fixed point inside every likelihood evaluation. MPEC treats the value function as a free optimizer vector and imposes the Bellman equation as an equality constraint. SEES keeps the joint-optimization idea but replaces the full value vector $V \in \mathbb{R}^{|\mathcal{S}|}$ with

$$V_K(s;\alpha) = \psi_K(s)^\top \alpha,$$

and penalizes Bellman residuals instead of imposing exact equality.

This document is a known-truth validation tutorial. It does not use real data. The synthetic DGP supplies the true reward, transition law, value function, policy, and Type A/B/C counterfactual oracle objects. The validation question is whether the SEES implementation recovers these known objects on medium-scale finite panels while enforcing the Bellman equation tightly enough for structural use. The low-dimensional cell is a sanity check. The high-dimensional cell is the primary validation because it requires SEES to build its sieve from encoded state features rather than from a one-dimensional state index.

2 Source Paper

The source paper is Luo and Sang (2024). The paper proposes a sieve-based efficient estimator for structural dynamic models. The value function is approximated by basis functions, and the equilibrium condition is imposed through a penalty. The estimator jointly optimizes structural parameters and sieve coefficients:

$$(\hat{\theta}, \hat{\alpha}) = \arg\max_{\theta,\alpha} \ell_n(\theta,\alpha) - \omega_n \left\| V_K(\alpha) - T_\theta V_K(\alpha) \right\|^2.$$

The theoretical target is not a rough simulator fit. Under smoothness, identification, sieve-approximation, and penalty-rate conditions, the structural parameter estimator is consistent, asymptotically normal, and efficient. The finite discrete-state limiting case is important for this package: when the sieve has enough columns to span the finite-state value vector and the penalty is large, SEES approaches the MPEC formulation.

The paper's main warning is also practical. SEES needs a basis rich enough to represent the value function and a penalty large enough to enforce equilibrium. Too few basis functions create projection bias. Too small a penalty allows the likelihood to fit choices with a value approximation that does not solve the dynamic model.

3 When to Use SEES

Use SEES when the model is structural, the reward is parametric, transitions are known or first-stage estimated, and the state space is large enough that NFXP or MPEC are costly but still smooth enough for deterministic basis functions to approximate V. SEES is a structural approximation method, not a behavioral cloning method.

Do not use SEES as a black-box rescue for a non-identified DGP. If the reward features are state-only and identical across actions, if an action is missing, or if the exit/scale normalization is absent, the likelihood cannot identify the structural parameters. The known-truth gates below are intentionally hard: a non-smoke run must pass Bellman residual, standard-error, reward, value, policy, and counterfactual checks.

4 Model

The observed data are state-action-next-state trajectories

$$\{(s_{it}, a_{it}, s_{i,t+1}) : i = 1, \ldots, N,\ t = 1, \ldots, T\}.$$

Flow payoff is linear in known features:

$$u_\theta(s, a) = \phi(s, a)^\top \theta.$$

Transitions are

$$P_a(s, s') = \Pr(s_{t+1} = s' \mid s_t = s, a_t = a),$$

the discount factor is β, and the type-I extreme-value shock scale is σ. Given V, the choice-specific value is

$$Q_\theta(s, a; V) = u_\theta(s, a) + \beta \sum_{s'} P_a(s, s') V(s').$$

The logit policy implied by (θ, V) is

$$\pi_{\theta,V}(a \mid s) = \frac{\exp(Q_\theta(s, a; V)/\sigma)}{\sum_b \exp(Q_\theta(s, b; V)/\sigma)}.$$

The soft Bellman operator is

$$T_\theta V(s) = \sigma \log \sum_a \exp\left(\frac{u_\theta(s, a) + \beta \sum_{s'} P_a(s, s') V(s')}{\sigma}\right).$$

5 SEES Estimator

Let $\Psi \in \mathbb{R}^{|\mathcal{S}| \times K}$ be the sieve basis matrix and $V_K = \Psi\alpha$. The implementation supports B-splines, polynomials, and Fourier bases. The known-truth validation uses B-splines because Luo and Sang use spline sieves as the paper's default smooth basis.

The penalized criterion is

$$\mathcal{Q}_n(\theta, \alpha) = \sum_{i,t} \log \pi_{\theta,\Psi\alpha}(a_{it} \mid s_{it}) - \omega \sum_s \left[\Psi_s \alpha - T_\theta(\Psi\alpha)(s)\right]^2.$$

The optimizer variable is $x = (\theta, \alpha)$. There is no nested value iteration. The package uses JAX automatic differentiation and L-BFGS-B over the joint vector. The reported log likelihood is the unpenalized choice log likelihood at the final $(\hat{\theta}, \hat{\alpha})$; the Bellman violation is reported separately and is part of the hard validation gate.

5.1 Standard Errors

Let the Hessian of the penalized objective in (θ, α) blocks be

$$H = \begin{pmatrix} H_{\theta\theta} & H_{\theta\alpha} \\ H_{\alpha\theta} & H_{\alpha\alpha} \end{pmatrix}.$$

SEES treats α as a nuisance approximation coefficient. The marginal information for θ uses the Schur complement:

$$\tilde{H}_\theta = H_{\theta\theta} - H_{\theta\alpha} H_{\alpha\alpha}^{-1} H_{\alpha\theta}.$$

The validation run computes this marginal Hessian and requires finite standard errors. A run with good point estimates but singular standard errors is not treated as a completed structural validation.

6 Identification Checks

1. **Reward rank.** The action-dependent feature matrix must have full column rank after dropping zero rows.
2. **Action support.** Each action must appear in the finite panel so the likelihood can discipline payoff differences.
3. **State coverage.** The canonical tabular validation observes every state in the regular state space.
4. **Normalization.** The reward scale is fixed at $\sigma = 1$, and the synthetic DGP uses an exit action and absorbing state as a zero-reward anchor.
5. **Basis adequacy.** The sieve must be rich enough for the DGP. In the low-dimensional sanity cell, $K = |\mathcal{S}| = 21$, so the B-spline basis spans the 21-state value vector. In the high-dimensional primary cell, SEES uses `problem.state_encoder` to build a full-rank encoded-state basis with $K = |\mathcal{S}| = 81$.
6. **Penalty strength.** The Bellman residual is not cosmetic. The non-smoke validation gates on the final residual and fails if the penalty allows V_K to drift away from $T_\theta V_K$.

7 Known-Truth DGP

The SEES primer runs two canonical cells. The first is `canonical_low_action`, the low-dimensional action-dependent structural benchmark used as a sanity check. The second is `canonical_high_action`, the high-dimensional state and reward-feature stress benchmark used as the primary SEES validation.

Table 1: Canonical SEES DGP cells.

Quantity	Low-dimensional	High-dimensional
Regular states	20	80
Absorbing states	1	1
Total states	21	81
State features	2	16
Reward parameters	4	32
Actions	3	3
Exit action	2	2
Discount factor	0.95	0.95
Shock scale	1.0	1.0
Reward mode	action-dependent	action-dependent
Simulated individuals	2,000	2,000
Periods per individual	80	80
Observations	160,000	160,000

In the low-dimensional sanity cell, the action-dependent features are

$$\phi(s,a) = \begin{cases} (1, x_s, 0, 0) & a = 0, \\ (0, 0, 1, x_s) & a = 1, \\ (0, 0, 0, 0) & a = 2. \end{cases}$$

The exit action has zero reward and sends the process to the absorbing state. The true parameter vector is:

Table 2: True reward parameters.

Parameter	Truth
`action_0_intercept`	0.10
`action_0_progress`	0.50
`action_1_intercept`	0.00
`action_1_progress`	-0.20

8 Pre-Estimation Diagnostics

Run diagnostics before fitting. For SEES, basis and penalty choices are only meaningful after identification checks pass.

```
from experiments.known_truth import (
    SimulationConfig,
    build_known_truth_dgp,
    get_cell,
    run_pre_estimation_diagnostics,
    simulate_known_truth_panel,
)

cell = get_cell("canonical_high_action")
dgp = build_known_truth_dgp(cell.dgp_config)
panel = simulate_known_truth_panel(
    dgp,
    cell.simulation_config,
)
diagnostics = run_pre_estimation_diagnostics(dgp, panel)

assert diagnostics.passed
assert diagnostics.feature_rank == diagnostics.num_features
assert diagnostics.is_action_dependent
assert diagnostics.observed_states == diagnostics.num_states
```

9 Run the Tutorial

From the repository root:

```
PYTHONPATH=src:. python -m experiments.known_truth \
    --estimator SEES \
    --cell-id canonical_high_action \
    --output-dir outputs/known_truth \
    --show-progress \
    --verbose
```

The command builds the DGP, simulates the panel with `tqdm` progress, runs pre-estimation diagnostics, fits SEES with Schur-complement standard errors, evaluates reward/value/Q/policy recovery, solves Type A/B/C counterfactual oracles, writes one `result.json`, and raises if any non-smoke hard gate fails.

To regenerate the full primer tables for both cells:

```
PYTHONPATH=src:. python papers/econirl_package/primers/sees/sees_run.py
```

Direct estimator usage:

```
from experiments.known_truth import (
    build_known_truth_dgp,
    get_cell,
    simulate_known_truth_panel,
)
from econirl.estimation.sees import SEESEstimator

cell = get_cell("canonical_low_action")
dgp = build_known_truth_dgp(cell.dgp_config)
panel = simulate_known_truth_panel(dgp, cell.simulation_config)

estimator = SEESEstimator(
```

```
    basis_type="bspline",
    basis_dim=dgp.problem.num_states,
    penalty_weight=100.0,
    max_iter=1000,
    tol=1e-7,
    compute_se=True,
    verbose=True,
)

summary = estimator.estimate(
    panel=panel,
    utility=dgp.utility(),
    problem=dgp.problem,
    transitions=dgp.transitions,
)
```

For the high-dimensional validation, the known-truth harness uses `basis_dim=dgp.problem.num_states`, an encoded-state basis selected automatically from `problem.state_encoder`, and a stronger Bellman penalty. The deterministic starting point is near the truth because this hard validation checks the SEES basis and Bellman formulation, not global discovery from an arbitrary 32-parameter start.

10 Generated Validation Results

This section is generated by `sees_run.py` from the `experiments.known_truth` harness. Hard gates are enforced. The low-dimensional cell is a sanity check for the historical index-based sieve; the high-dimensional cell is the main SEES validation because it uses the encoded-state sieve.

10.1 Validation Design

Both cells use 2,000 simulated individuals, 80 periods per individual, known transitions, three actions, homogeneous rewards, and the shared Type A/B/C counterfactual oracles.

Table 3: SEES known-truth validation cells.

Cell	Role	States	State dim.	Reward params	Basis	Penalty	Gates
Low-dimensional	sanity	21	2	4	state_index, K=21	100	11/11
High-dimensional	primary	81	16	32	encoded_state, K=81	10000	11/11

The high-dimensional DGP passes pre-estimation checks before SEES is fit: feature rank is 32/32, the reward-feature condition number is 1.377, all 81 states are observed, and state-action coverage is 0.959. The encoded-state basis uses 16 state features and has numerical rank 81.

10.2 Fit Summary

Table 4: SEES fit summary.

Cell	Optimizer flag	Iter.	Bellman max	Bellman RMSE	Log likelihood
`canonical_low_action`	false	181	5.83×10^{-5}	2.74×10^{-5}	-174875.8
`canonical_high_action`	false	6	3.08×10^{-6}	6.80×10^{-7}	-160289.8

The optimizer flag is reported exactly as returned by L-BFGS-B. The hard SEES validation gate is the Bellman residual, finite standard errors, known-truth recovery, and counterfactual regret.

10.3 Main Recovery Numbers

Table 5: Core recovery metrics.

Metric	Gate	Low-dimensional	High-dimensional
Bellman violation	≤ 0.05	0.000058	0.000003
Parameter cosine similarity	≥ 0.99	0.999146	0.999955
Parameter relative RMSE	≤ 0.15	0.059671	0.009528
Reward RMSE	≤ 0.03	0.008988	0.004432
Policy total variation	≤ 0.02	0.005179	0.002117
Value RMSE	≤ 0.10	0.017591	0.037836
Q RMSE	≤ 0.10	0.020514	0.031480

10.4 Counterfactual Recovery

Each counterfactual gate requires regret below 0.01.

Table 6: Counterfactual regret.

Counterfactual	Low-dimensional	High-dimensional	Interpretation
Type A	0.000179	0.000113	reward/state shift
Type B	0.000299	0.000183	transition change
Type C	0.000071	0.000014	action restriction

Exact parameter vectors, standard errors, log-likelihoods, policy distances, basis metadata, and all gate values are written to `sees_results.json`. The PDF reports the decision-critical values rather than printing the full 32-parameter high-dimensional vector.

10.5 Gate Audit

Table 7: Hard-gate audit by cell.

Cell	Passed gates	Failed gates
`canonical_low_action`	11	0
`canonical_high_action`	11	0

11 Debugging Gaps

If SEES fails to recover truth, debug in this order.

1. **Feature rank first.** If the reward design is not identified, larger sieves and larger penalties will only produce sharper nonsense.
2. **Basis dimension.** Low K is the standard finite-sample SEES failure mode. On this DGP, $K = 8$ recovers the policy but leaves value and parameter recovery outside the hard structural gates. The validation uses $K = 21$, equal to the finite-state value dimension.
3. **Penalty weight.** The penalty must make the Bellman residual small. A high likelihood with a loose Bellman residual is not a structural fit.
4. **Standard errors.** Non-finite Schur-complement standard errors mean the nuisance block is singular or nearly singular. Do not publish that run as a full SEES validation.

5. **Optimizer flag.** The package reports the L-BFGS-B absolute-gradient flag exactly. For SEES, the known-truth gate is the structural residual plus recovery metrics, because the sample-summed penalized objective can plateau with a large absolute gradient while the structural objects are already recovered.
6. **Scale and anchor.** If the logit scale or exit-action anchor is changed, reward levels can drift.
7. **Soft gates.** Do not accept a non-smoke run that only warns. Known-truth validation should pass hard gates or raise.

12 Practical Defaults

Recommended non-smoke validation settings for the low-dimensional sanity cell:

```
SEESEstimator(
    basis_type="bspline",
    basis_dim=dgp.problem.num_states,
    penalty_weight=100.0,
    max_iter=1000,
    tol=1e-7,
    compute_se=True,
)
```

Recommended high-dimensional validation settings:

```
SEESEstimator(
    basis_type="bspline",
    basis_dim=dgp.problem.num_states,
    penalty_weight=10000.0,
    max_iter=1000,
    tol=1e-7,
    compute_se=True,
)
```

Use smoke settings only for code-path tests. A smoke test shows that the estimator runs and returns finite objects; it is not evidence of recovery.

13 Implementation Paths

- Estimator: `src/econirl/estimation/sees.py`
- Known-truth harness: `experiments/known_truth.py`
- Component tests: `tests/test_sees_known_truth_components.py`
- Fast validation tests: `tests/test_known_truth.py`
- Master plan: `papers/econirl_package_jss/plans/README.md`
- SEES alignment audit: `papers/econirl_package_jss/plans/alignment/13_sees.md`

14 References

References

Yao Luo and Peijun Sang. Efficient estimation of structural models via sieves. Working Paper, University of Toronto, 2024.